

PyBaMM

Training day 1: PyBaMM basics and real-world simulations

PyBaMM Conference 2026 · Ann Arbor, Michigan · 15 October 2026

Today's session

30 min

Talk: what PyBaMM is, how a simulation works, and how to run realistic experiments

60 min

Self-guided notebooks, with the PyBaMM team on hand to help

- 01 Getting started — models, parameters, experiments, outputs
- 02 Power vs energy cells — C-rate sweeps and electrode thickness
- 03 Long experiments — hundreds of cycles, summary variables
- 04 Reference performance tests — chaining ageing and RPT protocols
- 05 Custom experiments — custom terminations and control

Set up before we start

```
git clone https://github.com/pybamm-team/pybamm-con-2026
cd pybamm-con-2026
pip install -r requirements.txt
jupyter lab
```



Pinned versions

pybamm 26.9.0.0 and pybamm-model-zoo 0.1.0, so every notebook runs as tested.

Stuck installing? Ask now: the notebooks need a working environment.

What is PyBaMM?

A framework

A computer algebra system for writing and solving differential equations. Models are symbolic expression trees.

A library

Battery models from SPM to DFN, with options for thermal, degradation and more, and dozens of published parameter sets.

Experiment tools

Lab-style protocols in plain English, long cycling, summary variables and plotting.

Every simulation is a pipeline



- Model: symbolic equations, with no numbers yet
- Parameter values replace symbols with numbers or functions
- Discretisation turns div and grad into matrices on a mesh: method of lines
- Solver (IDAKLU, the default) steps the resulting DAEs in time

Three lines to a simulation

```
import pybamm

model = pybamm.lithium_ion.DFN()
sim = pybamm.Simulation(model)
sim.solve([0, 3600])
sim.plot()
```

- Swap DFN for SPM or SPMe: same API, much faster
- Options change the physics: DFN({"thermal": "lumped"})
- No solver argument: IDAKLU is the default for every model

Models: pick the cheapest one that answers your question

SPM

One particle per electrode. Fast, good at low C-rates.

SPMe

SPM plus electrolyte. A good default for most rates.

DFN

Full porous-electrode model. The reference, and the slowest.

Options

thermal, SEI, lithium plating, particle cracking, loss of active material, particle size distributions…

Parameters

```
parameter_values =
pybamm.ParameterValues("Chen2020")
parameter_values.search("electrode thickness")

parameter_values["Current function [A]"] = 10
sim = pybamm.Simulation(
    model, parameter_values=parameter_values
)
```

- Dozens of published parameter sets
- Any value can be a number, a function, or data
- Names are the API: "Name [unit]"

Experiments in plain English

```
experiment = pybamm.Experiment(
    [
        (
            "Discharge at C/2 until 3.0V",
            "Rest for 1 hour",
            "Charge at 1C until 4.1V",
            "Hold at 4.1V until 50 mA",
        )
    ]
    * 2
)
sim = pybamm.Simulation(model,
experiment=experiment)
sol = sim.solve()
```

- Current, C-rate, power, voltage and resistance control
- Terminations on voltage, current or time
- Drive cycles from data with pybamm.step.current(data)

Reading the outputs

```
sol["Voltage [V]"](t)           # interpolate at times t [s]
sol.cycles[0].steps[1]          # one step of one cycle
sol.summary_variables           # per-cycle metrics
model.variables.search("temperature")
```

Call, don't index

Calling a variable interpolates it. The raw .data arrays are tied to the solver's internal time steps.

Cycles and steps

Experiments split the solution so each step can be analysed on its own.

Summary variables

Capacity, lithium inventory and active material per cycle, for degradation studies.

Doing real things with PyBaMM

Notebooks 02 to 05

02 · Power vs energy cells

Sweep C-rate 0.1C to 20C › **Energy and mean power per run** › **Halve electrode thickness** › **Compare the two cells**

- Thick electrodes store more energy but lose it fast at high rate
- Thin electrodes hold their capacity to much higher C-rates
- Exercise: define a cell mass and compute specific energy and power

03 & 04 · Long experiments and RPTs

```
experiment = pybamm.Experiment(
    [cycle] * 500, termination="80% capacity"
)
sol = sim.solve(save_at_cycles=5)
pybamm.plot_summary_variables(sol)

# chain protocols
rpt_sol = rpt_sim.solve(starting_solution=ageing_sol)
```

- Stop on capacity, not cycle count
- Save every nth cycle, or chosen ones
- Chain ageing, charge and RPT protocols with starting_solution

05 · Custom experiments

Custom terminations

Stop a step on any model variable, e.g. anode potential below 0.02 V to avoid plating.

Explicit control

Set the current as a function of other variables, e.g. constant power I = P / V.

Implicit control

Give an equation the current must satisfy, e.g. hold the anode potential constant.

Over to you

Open training/day_1 and start with 01-getting-started

Self-guided session

New to PyBaMM?

Work through 01-getting-started, then 02 power vs energy.

Used it before?

Jump to 03 to 05: long cycling, RPTs and custom experiments. Or bring your own problem.

Stuck?

Put your hand up. Docs: docs.pybamm.org. Issues and discussions: github.com/pybamm-team/PyBaMM

Tomorrow: the new PyBaMM model zoo, writing your own model, input parameters and solvers